HireDesk

The complete code guide for a junior full-stack developer

What was built, how every part works, and how to explain it

Covers: the product · PRD, SRS and ERD · every tool and library · backend and frontend code file by file · database and migrations · security · tests · deployment on Vercel and Neon · CI/CD with GitHub Actions · the full change history · practice interview questions

Live site: https://hiredesk-app.vercel.app

API and interactive docs: https://hiredesk-api.vercel.app/docs

Repository: github.com/Daveoski/hiredesk_official

Prepared: 11 October 2026

Contents

1. How to use this guide 5
2. What HireDesk is 6
    2.1 The people who use it 6
    2.2 The journey from job to hire 6
    2.3 The pipeline at a glance 7
3. How a web app works (the basics) 8
    3.1 Key ideas 8
4. Glossary: every tool and library used 10
    4.1 Languages 10
    4.2 Frontend 10
    4.3 Backend 11
    4.4 Data, hosting and services 12
5. The PRD (Product Requirements Document) 14
    5.1 What a PRD is 14
    5.2 What HireDesk's PRD says 14
    5.3 What has changed since the PRD was written 15
6. The SRS (Software Requirements Specification) 16
    6.1 What an SRS is 16
    6.2 Functional requirements, in plain words 16
    6.3 Non-functional requirements 16
    6.4 Changes since the SRS was written 17
7. The ERD (Entity Relationship Diagram) 18
    7.1 What an ERD is 18
    7.2 The tables 18
    7.3 Integrity rules the database itself enforces 19
    7.4 The architecture diagram 20
8. The repository layout 21
9. Configuration and environment variables 22
10. Backend: how a request travels through FastAPI 24
    10.1 main.py – the entry point 24
    10.2 Dependency injection 25
11. Backend: the database layer 26
    11.1 The session 26
    11.2 Shared column helpers 26
    11.3 Models 27
    11.4 Schemas vs models 27
12. Backend: security – passwords, tokens and permissions 28
    12.1 Password hashing 28
    12.2 JWT login tokens 28
    12.3 Authentication and role dependencies 28
    12.4 Permissions: who can see which rows 29
13. Backend: sign-up, login, Google and invitations (auth/router.py) 30

13.1 Register and login 30
13.2 Google sign-in 30
13.3 Changing a password 30
13.4 Accepting an invitation 31
14. Backend: the team and invitations (users/router.py) 32
15. Backend: jobs, public applications and file uploads 33
15.1 Jobs 33
15.2 The public side (no login) 33
15.3 Uploading large files on a serverless host 33
16. Backend: the hiring pipeline (candidates) 35
16.1 The state machine 35
16.2 Changing a stage safely (row locking) 35
16.3 The candidate endpoints 35
17. Backend: interviews and the no-double-booking rule 37
17.1 Two steps: assign, then schedule 37
17.2 The exclusion constraint 37
18. Backend: scorecards and fair feedback 39
18.1 Submitting a scorecard 39
18.2 The privacy rule (avoiding anchoring bias) 39
19. Backend: email, notifications and reports 40
19.1 Sending email 40
19.2 Background tasks 40
19.3 Admin notifications 40
19.4 The progress report 40
20. Backend: migrations and tests 41
20.1 Alembic migrations 41
20.2 The tests 41
21. Frontend: Next.js basics you need 43
21.1 Components and JSX 43
21.2 The App Router: folders are URLs 43
22. Frontend: talking to the backend 45
22.1 The API client (lib/api.ts) 45
22.2 The auth store (stores/auth-store.ts) 45
22.3 Server data with TanStack Query (lib/queries.ts) 45
22.4 Clearing data on logout (privacy fix) 46
23. Frontend: types, validation and forms 47
24. Frontend: the public pages 48
24.1 Landing page (app/page.tsx) 48
24.2 Login, register and Google sign-in 48
24.3 Accepting an invitation (app/accept-invite) 48
24.4 Applying for a job (app/apply/[jobId]) 48
25. Frontend: the signed-in app 49
25.1 Protected layout and app shell 49
25.2 Pages 49
25.3 Reusable components 49

26. Frontend: the dashboards 51
26.1 Accessible colours 51
27. Frontend: styling, errors and security headers 52
27.1 Tailwind and design tokens 52
27.2 Error and 404 pages 52
27.3 Security headers (next.config.ts) 52
28. Deployment: how the app is hosted 53
28.1 What each deployment file does 53
28.2 Problems solved during deployment 53
29. CI/CD with GitHub Actions 55
30. External services setup 57
30.1 Google sign-in 57
30.2 Email 57
30.3 Cloudinary 57
31. What happened: the full timeline of the work 58
31.1 First review and bug fixes 58
31.2 Google sign-in for admins and hiring managers 58
31.3 Password change, personal dashboards, admin emails and reports 58
31.4 Dashboard redesign 58
31.5 Deployment 58
31.6 Google origin and invitation emails 59
31.7 Full code review 59
32. The hard problems and how they are solved 60
33. Practice questions and model answers 61
34. Known limitations and next steps 64
Appendix A. Every API endpoint 65
Appendix B. HTTP status codes used 67
Appendix C. Useful commands 68

If the page numbers above look empty, right-click the table in Word and choose "Update field".

1. How to use this guide

This guide explains HireDesk from the ground up: what the app does, the planning documents (PRD, SRS, ERD), every tool and library, how each part of the code works, what was changed and fixed, how it is deployed, and the questions you are likely to be asked about it.

It is written for a junior full-stack developer. Every technical word is defined the first time it appears, and Section 4 is a glossary you can come back to. Code is shown in grey boxes; file paths are shown like `backend/app/main.py`.

- **Sections 2–4**: the big picture, how web apps work, and a glossary of every tool. Read these first.
- **Sections 5–7**: the planning documents: PRD, SRS, ERD and architecture.
- **Sections 8–9**: the folder layout and configuration.
- **Sections 10–20**: the backend (Python/FastAPI) file by file.
- **Sections 21–27**: the frontend (Next.js/React) file by file.
- **Sections 28–30**: deployment, CI/CD and the external services.
- **Section 31**: what happened, in order, during the build and review.
- **Section 32**: the hard problems and how they are solved.
- **Section 33**: practice questions with model answers.
- **Section 34 and appendices**: known limitations, every API endpoint, HTTP status codes and useful commands.

> **Tip:** When someone asks you a question about the project, first say *what problem* the code solves, then *where* it lives, then *how* it works. This guide follows that order for every feature.

2. What HireDesk is

HireDesk is a hiring workspace (an *applicant tracking system*, or ATS) for small companies. It takes a job from "we need someone" to "we hired them". The whole team works in one place, and every candidate is kept informed by email.

Live site: **https://hiredesk-app.vercel.app** · API: **https://hiredesk-api.vercel.app** (interactive API docs at `/docs`).

2.1 The people who use it

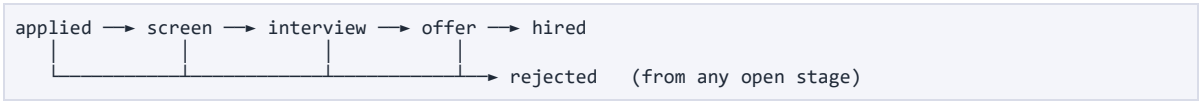
Role	Stored as	What they do
Company admin	`company_admin`	Creates the company account, invites teammates, watches progress across all jobs, receives progress emails and weekly reports. Does **not** open individual CVs.
Hiring manager	`hiring_manager`	Creates jobs, reviews applicants for their own jobs, moves candidates through stages, assigns interviewers, records an assessment, makes the final hire/reject decision.
Interviewer	`interviewer`	Sees only candidates assigned to them, picks the interview time and format, submits a scorecard.
Candidate	(no account)	Applies through a public link with a CV and documents, and receives emails at every step.

2.2 The journey from job to hire

1. An **admin signs up** (email + password, or Google). This creates a *company* and makes them its admin.
2. The admin **invites** hiring managers and interviewers. Each gets an email with a one-time "Join the team" link that expires in 7 days.
3. A **hiring manager creates a job** (draft), then **opens** it. Opening produces a public apply link.
4. **Candidates apply**: name, email, phone, qualifications, optional salary and cover letter, a CV (≤ 5 MB) and up to 6 documents (≤ 10 MB each).
5. Each candidate moves through a fixed **pipeline**: Applied → Screen → Interview → Offer → Hired, or Rejected from any open stage. Every move is recorded.
6. The hiring manager **assigns an interviewer**. The interviewer **schedules** a time and format (video link or in person); the candidate is emailed.
7. After the interview starts, the interviewer submits a **scorecard**: criteria rated 1–5, comments, and a recommendation.
8. The hiring manager reviews scorecards and records an **assessment** (match score 0–100 and notes), then **hires or rejects**.
9. The **admin** gets an email for each important event, sees a company-wide **dashboard**, and receives a **Monday progress report**.

2.3 The pipeline at a glance

Allowed stage moves (backend/app/candidates/service.py)



3. How a web app works (the basics)

Before reading the code, it helps to know the pieces every web app has. HireDesk is a classic **three-tier** app: a frontend, a backend, and a database.

The three tiers of HireDesk

```
Browser (the user)
     │  loads pages
     ▼
FRONTEND  ── Next.js / React, runs in the browser ──  hiredesk-app.vercel.app
     │  HTTP requests with JSON (fetch)
     ▼
BACKEND   ── FastAPI / Python, runs on a server ────  hiredesk-api.vercel.app
     │  SQL queries
     ▼
DATABASE  ── PostgreSQL on Neon ────────────────────  stores everything
     +  external services: Google (sign-in), Cloudinary (files), Resend/Gmail (email)
```

3.1 Key ideas

Term	Meaning	In HireDesk
Client / server	The client asks, the server answers. A browser is a client; the API is a server.	The Next.js app (client) calls the FastAPI app (server).
Frontend	Everything the user sees and clicks.	`frontend/` (Next.js, React, TypeScript).
Backend	Code on a server that applies rules, checks permissions and talks to the database.	`backend/` (Python, FastAPI).
API	Application Programming Interface: the list of requests a server accepts and what it returns.	35 endpoints such as `POST /jobs` (see Appendix A).
Endpoint / route	One URL + method the API answers, e.g. `GET /jobs`.	Defined with `@router.get(...)` in Python.
HTTP	The protocol browsers and servers use to talk.	Every call between frontend and backend.
HTTP method	The verb of a request: GET (read), POST (create/act), PUT (replace), PATCH (change part), DELETE (remove).	`PATCH /applications/{id}/stage` moves a candidate.
Status code	A number in every response: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 410 Gone, 422 Invalid input, 5xx server problems.	See Appendix B.
JSON	JavaScript Object Notation: a text format for data, e.g. `{"title": "Designer"}`.	Most request and response bodies.
Database	A program that stores data safely and answers queries.	PostgreSQL, hosted by Neon.
SQL	Structured Query Language, used to read and	Generated mostly by

Term	Meaning	In HireDesk
	write relational databases.	SQLAlchemy/SQLModel.
Table, row, column	A table is like a spreadsheet: columns are fields, rows are records.	`users`, `jobs`, `applications`, …
Authentication	Proving *who* you are (logging in).	Password or Google, then a JWT token.
Authorization	Deciding what you are *allowed* to do.	Roles + company scope (Section 12).
Environment variable	A setting given to a program from outside its code (e.g. a password).	`DATABASE_URL`, `JWT_SECRET`, … (Section 9).

4. Glossary: every tool and library used

This is the full list of technologies in the project, what each one is, and why it is used. Versions come from `frontend/package.json`, `backend/requirements.txt` and `pyproject.toml`.

4.1 Languages

Name	What it is	Why HireDesk uses it
Python 3.12	A general-purpose programming language known for readable code.	The whole backend is written in Python.
TypeScript	JavaScript with *types*: you declare what shape data has, and mistakes are caught before running.	The whole frontend. `tsc --noEmit` checks types.
JavaScript	The language browsers run. TypeScript compiles to it.	What actually runs in the browser.
SQL	The language of relational databases.	Migrations and a few raw statements (e.g. the exclusion constraint).
HTML / CSS	Page structure / page styling.	Produced by React components; CSS via Tailwind.
Bash / YAML	Shell scripting / a configuration format.	Setup wizard script; GitHub Actions workflows.

4.2 Frontend

Name	What it is	Where / why
Next.js 15	A React framework: routing by folders, building, optimisation, and server rendering.	`frontend/src/app/` – each folder with a `page.tsx` is a URL.
React 19	A library for building user interfaces from *components* (functions that return UI).	Every screen is made of components.
Tailwind CSS 4	Utility-first CSS: style by adding small classes like `p-4 rounded-lg`.	All styling; colours are defined as tokens in `globals.css`.
shadcn/ui + Radix UI	Copy-in, accessible UI building blocks (dialogs, dropdowns, tabs).	`frontend/src/components/ui/`.
TanStack Query (React Query) 5	Fetches server data, caches it, refetches and tracks loading/error states.	`lib/queries.ts` – `useJobs()`, `useApplications()`, …
Zustand 5	A tiny state-management library (shared memory for the app).	`stores/auth-store.ts` holds the token and user.
React Hook Form 7	Manages form state and validation efficiently.	Login, register, apply, job, schedule, scorecard forms.

Name	What it is	Where / why
Zod 3	Describes data rules (schemas) and validates input.	`lib/schemas.ts` – e.g. password at least 8 characters.
lucide-react	An icon set.	All icons.
sonner	Toast notifications (small pop-up messages).	"Interview scheduled", error messages.
clsx + tailwind-merge	Combine class names safely.	`cn()` in `lib/utils.ts`.
Google Identity Services	Google's browser script that shows the "Sign in with Google" button and returns an ID token.	`components/shared/google-sign-in-button.tsx`.
Bun	A fast JavaScript runtime and package manager (like npm).	Installs frontend packages; `bun.lock` records exact versions.

4.3 Backend

Name	What it is	Where / why
FastAPI	A Python web framework for building APIs. Uses type hints to validate input and auto-generate docs.	`backend/app/main.py` and every `router.py`.
Uvicorn	An ASGI server: the program that runs a FastAPI app and listens for HTTP requests.	Local development (`uvicorn app.main:app`).
ASGI	Asynchronous Server Gateway Interface: the standard that connects Python web apps to servers.	How Vercel runs FastAPI.
Pydantic 2	Validates and converts data using Python classes with type hints.	Request/response *schemas* (`schemas.py` files).
pydantic-settings	Reads configuration from environment variables into a Pydantic class.	`core/config.py`.
SQLModel	Combines Pydantic and SQLAlchemy: one class is both a table and a data model.	Every `models.py`.
SQLAlchemy	Python's main database toolkit and ORM.	Queries, sessions, transactions, row locks.
ORM	Object-Relational Mapper: work with database rows as Python objects instead of raw SQL.	`db.get(User, id)`, `select(Job).where(...)`.
Alembic	Database *migrations*: versioned scripts that change the database structure.	`backend/alembic/versions/0001…0005`.

Name	What it is	Where / why
psycopg 3	The driver that lets Python talk to PostgreSQL.	Used by SQLAlchemy (`postgresql+psycopg://`).
PyJWT	Creates and verifies JSON Web Tokens.	`core/security.py`.
pwdlib + Argon2	Password hashing. Argon2 is the current recommended algorithm.	`hash_password`, `verify_password`.
google-auth	Google's library to verify ID tokens on the server.	`/auth/google`.
python-multipart	Lets FastAPI read form posts and file uploads.	Login form, application form.
email-validator	Checks email addresses (via `pydantic[email]`).	`EmailStr`.
Cloudinary SDK	Uploads and manages files on Cloudinary.	`core/storage.py`.
Resend SDK	Sends email through the Resend service.	`core/email.py`.
smtplib	Python's built-in email sender over SMTP.	Gmail sending in `core/email.py`.
pytest	The Python testing framework.	`backend/tests/` – 73 tests.
httpx / TestClient	An HTTP client; FastAPI's TestClient uses it to call the API inside tests.	`client.post(...)` in tests.
uv	A fast Python package and project manager.	Root `pyproject.toml` + `uv.lock`.

4.4 Data, hosting and services

Name	What it is	Where / why
PostgreSQL	A powerful open-source relational database.	Stores companies, users, jobs, applications, …
Neon	A hosted (serverless) PostgreSQL service, added through Vercel's Marketplace. Replaced "Vercel Postgres".	The production database `hiredesk-db`.
Vercel	A cloud platform that builds and hosts web apps from a Git repository.	Two projects: `hiredesk-official` (site) and `hiredesk-api` (backend).
Serverless function	Code that runs on demand per request instead of on an always-on server.	Vercel runs the FastAPI app this way.

Name	What it is	Where / why
Cloudinary	A cloud service for storing and serving files.	CVs and supporting documents.
Resend	An email-sending service with an API.	Default email provider.
SMTP / Gmail app password	SMTP is the standard protocol for sending email. An *app password* lets an app log in to Gmail without your real password.	Optional email provider that can reach any inbox without a domain.
Google Cloud OAuth client	A registration in Google Cloud that allows a website to use "Sign in with Google".	Client ID `285740906915-…`; allowed origin `https://hiredesk-app.vercel.app`.
Git / GitHub	Git tracks code changes; GitHub hosts the repository online.	`github.com/Daveoski/hiredesk_official`.
GitHub Actions	GitHub's automation: runs workflows on events like a push.	`.github/workflows/` – tests, builds, migrations, weekly reports.
CI/CD	Continuous Integration (automatically test every change) / Continuous Delivery (automatically deploy it).	GitHub Actions (CI) + Vercel auto-deploy (CD).

5. The PRD (Product Requirements Document)

5.1 What a PRD is

A **Product Requirements Document** describes *what* a product must do and *why*, from the user's point of view. It names the users, the problems, the goals, what is in and out of scope, and acceptance criteria (how you know a requirement is met). It does not say how to code it.

HireDesk's PRD is `specs/prd-hiredesk.md`. It is an "as-built baseline": written to describe the product the code actually supports.

5.2 What HireDesk's PRD says

Problem: hiring teams need one place to review applicants, coordinate interviews, record assessments and communicate outcomes, while candidate data is visible only to the right people.

Goals:

- Publish roles and receive applications with CVs and supporting information.
- Give hiring managers a clear, step-by-step pipeline and decision workflow.
- Let interviewers schedule meetings and submit *independent* scorecards.
- Give admins team administration and outcome notifications without access to applicant records.
- Keep an auditable stage history; enforce company and role access in the backend.

Requirements (PR-1 to PR-9):

ID	Requirement	Acceptance (how we know it works)
PR-1	Company workspace and identity (email/password or Google).	Registration creates one company + one admin; duplicate emails rejected; bad Google tokens rejected.
PR-2	Team invitations (hiring manager or interviewer).	One-time, expires in 7 days; accepting keeps the invited company and role.
PR-3	Job management and public listing.	Draft by default; only open jobs are public; invalid salary ranges rejected.
PR-4	Candidate applications without an account.	CV PDF/DOC/DOCX ≤ 5 MB; ≤ 6 documents ≤ 10 MB; one application per email per job.
PR-5	Stage-based review.	Only valid moves; every move records from, to, who, when.
PR-6	Assessment and final decision.	Only hiring managers; hire only from Offer; admin gets a summary email.
PR-7	Interview coordination.	Same-company interviewers only; no overlapping interviews; cancelled slots reusable.
PR-8	Interview scorecards.	One per interview; 1–20 criteria rated 1–5; cannot resubmit.
PR-9	Role-appropriate access.	Out-of-scope data is never revealed; wrong role gets

ID	Requirement	Acceptance (how we know it works)
		403.

Out of scope (deliberately not built): candidate accounts, custom pipelines, calendar sync, AI CV ranking, billing, multi-language, password reset and MFA.

5.3 What has changed since the PRD was written

The PRD lists some *open questions*. This week's work answered several of them:

PRD statement	Now
"Google sign-in does not consume a pending invitation; it creates a new company-admin."	Fixed. Google sign-in now joins the inviting company with the invited role. A new company is only created from the Register page.
"Analytics dashboards are out of scope."	Each role now has its own dashboard; admins get a company-wide progress report and weekly email.
Admins receive only assessment and hire emails.	Admins are now emailed for stage moves, rejections, interviews (assigned, scheduled, cancelled), scorecards and new teammates.
No password change.	Every user can change their password on the Account page.

6. The SRS (Software Requirements Specification)

6.1 What an SRS is

A **Software Requirements Specification** is more technical than a PRD. It lists exactly how the system must behave, so engineers and testers can check it. It has two kinds of requirement:

- **Functional requirements (FR)** – what the system does, e.g. "FR-013: hiring is only allowed from the Offer stage".
- **Non-functional requirements (NFR)** – qualities of the system: security, reliability, performance, e.g. "passwords are stored hashed".

HireDesk's SRS is `specs/srs-hiredesk.md`. Each requirement is marked *Implemented*, *Configuration-dependent* (needs external credentials) or *Open*.

6.2 Functional requirements, in plain words

ID	Plain-English meaning
FR-001	Registering creates a company and its first admin. Emails are stored in lowercase and must be unique.
FR-002	Correct login returns a signed token; `/auth/me` returns who you are.
FR-003/004	Google sign-in: the server verifies Google's token and that the email is verified, then finds or creates the user.
FR-005	Admins invite hiring managers/interviewers; invites are hashed, expire in 7 days, and work once.
FR-006/007	Hiring managers create/update jobs (draft/open/closed); only open jobs are public.
FR-008–010	Candidates apply without an account; one application per email per job; file type and size rules.
FR-011	The fixed pipeline; every move adds a stage-history row.
FR-012/013	Assessment (score 0–100, notes ≤ 5,000 characters); final decision; hire only from Offer.
FR-014–017	Assign interviewers from the same company; the interviewer schedules a future time (15–240 minutes); no overlaps; candidate emailed.
FR-018–020	One scorecard per interview, 1–20 unique criteria rated 1–5; immutable after submission; the candidate's overall recommendation is computed (any "not recommend" wins; all "recommend" → recommend; otherwise maybe).
FR-021/022	Managers see all scorecards and the average; interviewers see others' scorecards only after submitting their own.
FR-023	Candidate and admin emails are sent when email is configured.

6.3 Non-functional requirements

- **Security**: invalid tokens get 401; authorization is enforced on the server (hiding a button is not security); Google identity only from a verified token; invitation secrets single-use, time-limited and hashed; passwords hashed.
- **Reliability**: stage changes are serialised per application; overlap prevention is enforced by PostgreSQL; duplicate scorecard submissions are blocked; email failures never fail a request.
- **API behaviour**: validation errors return 422, conflicts 409; application lists support filters, sorting and pagination (limit 1–200).

6.4 Changes since the SRS was written

SRS says	Now
NFR-API-3: tokens last 60 minutes.	Default is now **480 minutes** (one working day); expired sessions return you to the same page after login.
FR-018: submit a scorecard once the interview is *scheduled*.	Now only once the interview has **started** (bug fix).
Uploads go through the API.	The apply page uploads files **directly to Cloudinary** with a signed permission, because Vercel limits request bodies to 4.5 MB. The old way still works.
Resend only.	Email can go through **Resend or Gmail (SMTP)**.
API surface table.	New endpoints: password change, invitation list/resend/revoke, upload signing, progress reports (Appendix A has all 35).

7. The ERD (Entity Relationship Diagram)

7.1 What an ERD is

An **Entity Relationship Diagram** is a picture of the database: the *entities* (tables), their *attributes* (columns), and how they relate. The file is `docs/architecture/hiredesk-erd-and-architecture.md`; it is written in **Mermaid**, a text format that GitHub draws as a diagram.

Symbol / word	Meaning
Entity	A thing we store, i.e. a table (e.g. JOBS).
Attribute	A column (e.g. `title`).
PK – Primary Key	The column that uniquely identifies each row. Here always `id`, a UUID.
FK – Foreign Key	A column that points to a row in another table, e.g. `jobs.company_id → companies.id`.
UK – Unique Key	No two rows may share this value, e.g. `users.email`.
UUID	A 128-bit random identifier like `9f1c…`. Hard to guess, safe to expose in URLs.
`||--o{`	Crow's-foot notation: "exactly one" on the left, "zero or many" on the right. One company has many users.
`||--||`	Exactly one to exactly one: one interview has one scorecard.
`o|--o{`	Zero or one to many: a job may have no hiring manager (nullable FK).
timestamptz	A PostgreSQL date-time *with time zone*, so times are never ambiguous.
enum	A column restricted to a fixed list of values, e.g. role ∈ {company_admin, hiring_manager, interviewer}.

7.2 The tables

Relationships from the ERD (read "one company owns zero or many users")

```
COMPANIES ||--o{ USERS          : owns
COMPANIES ||--o{ JOBS           : owns
COMPANIES ||--o{ INVITATIONS    : issues
USERS     o|--o{ JOBS           : manages (hiring_manager_id)
USERS     ||--o{ INVITATIONS    : invites (invited_by_id)
JOBS      ||--o{ APPLICATIONS   : receives
APPLICATIONS ||--o{ STAGE_HISTORY : records
USERS     o|--o{ STAGE_HISTORY  : changes (changed_by_id)
APPLICATIONS ||--o{ INTERVIEWS  : has
USERS     ||--o{ INTERVIEWS     : conducts (interviewer_id)
INTERVIEWS ||--|| SCORECARDS    : has
SCORECARDS ||--o{ SCORECARD_RATINGS : contains
```

Table	Important columns	Notes
companies	id, name	Every other business row belongs to a company, directly or through

Table	Important columns	Notes
		a job.
users	id, company_id (FK), email (unique), full_name, hashed_password, password_login_enabled, role	`password_login_enabled` was added in migration 0004: false for Google-only accounts until they set a password.
invitations	id, company_id, invited_by_id, email, full_name, role, token_hash (unique), expires_at, accepted_at, created_at	Only the SHA-256 *hash* of the invite token is stored.
jobs	id, company_id, hiring_manager_id, title, description, qualification_requirements, salary_min/max, status, created_at	status ∈ draft / open / closed.
applications	id, job_id, full_name, email, phone, cv_url, candidate_qualifications, expected_salary, match_score, manager_notes, interviewer_recommendation, supporting_documents (JSON), cover_letter, stage, created_at	There is no separate candidate table: the candidate lives on the application. Unique (job_id, email).
stage_history	id, application_id, from_stage, to_stage, changed_by_id, changed_at	First row has from_stage = NULL (the application itself).
interviews	id, application_id, interviewer_id, starts_at, ends_at, status, meeting_type, meeting_url, location	status ∈ assigned / scheduled / completed / cancelled. Times are empty until scheduled.
scorecards	id, interview_id (unique), status, recommendation, submitted_at	Exactly one per interview, created as *pending* when the interview is assigned.
scorecard_ratings	scorecard_id + criterion (composite PK), rating, comment	A CHECK constraint keeps rating between 1 and 5.

7.3 Integrity rules the database itself enforces

- **Unique email** per user (`ix_users_email`).
- **One application per email per job** (`uq_applications_job_id_email`).
- **No overlapping interviews** for one interviewer (`no_overlapping_interviews`, an *exclusion constraint* – see Section 17).
- **Ratings 1–5** (`ck_scorecard_ratings_rating`) and valid statuses/recommendations (CHECK constraints).
- **Foreign keys** so a row can never point to something that does not exist.

> **Why put rules in the database?** Code checks can be bypassed by two requests arriving at the same instant. A database constraint is checked inside the database's own transaction, so it can never be broken – even under heavy load.

7.4 The architecture diagram

The same document has a **system context** diagram. In words: the candidate's browser and the team's browser both load the Next.js frontend. The frontend calls the FastAPI backend over HTTPS, sending a JWT for private routes. The backend verifies Google ID tokens with Google, reads and writes PostgreSQL in transactions, uploads files to Cloudinary, and sends email through Resend (or Gmail). It also contains *sequence diagrams* for applying and for interviews/scorecards, and an **authorization matrix** (who may do what), which Section 12 explains in code.

The document's "Known gaps" list is partly out of date: invited users *can* now sign in with Google, and the production topology is now defined (Section 28).

8. The repository layout

A **repository** (repo) is the project folder tracked by Git. HireDesk is a *monorepo*: frontend and backend live together.

Folder map

```
hiredesk_official/
├── backend/                FastAPI app (deployed as Vercel project "hiredesk-api")
│   ├── app/
│   │   ├── main.py         creates the FastAPI app, CORS, includes every router
│   │   ├── core/           config, security (hashing/JWT), email, notifications, storage
│   │   ├── db/             database session and shared column helpers
│   │   ├── auth/           login, register, Google, invitations accept, permissions
│   │   ├── users/          User + Invitation models, team management
│   │   ├── companies/      Company model, /companies/me
│   │   ├── jobs/           Job model, job endpoints
│   │   ├── candidates/     Application + StageHistory, pipeline rules, public apply
│   │   ├── interviews/     Interview model, assign/schedule/cancel
│   │   ├── scorecards/     Scorecard + ratings, submit, privacy rule
│   │   └── reports/        progress report, report emails, weekly script
│   ├── alembic/versions/   database migrations 0001–0005
│   ├── tests/              73 pytest tests
│   ├── requirements.txt    runtime packages;  requirements-dev.txt adds pytest
│   ├── vercel.json         Vercel function settings
│   └── .env.example        every setting, documented
├── frontend/               Next.js app (deployed as Vercel project "hiredesk-official")
│   ├── src/app/            pages (the folder structure IS the URL structure)
│   ├── src/components/     ui/ (primitives), shared/, layout/, features/
│   ├── src/lib/            api client, data hooks, types, schemas, constants
│   ├── src/stores/         auth store
│   └── next.config.ts      security headers
├── specs/                  PRD and SRS
├── docs/architecture/      ERD and architecture
├── scripts/                setup wizard for Google sign-in + Gmail
├── .github/workflows/      Backend, Frontend and Progress-reports automation
└── README.md, WORK_LOG.md
```

Inside the backend, each *feature folder* has the same pattern, which makes it easy to find things:

File	Holds
models.py	Database tables as SQLModel classes.
schemas.py	Pydantic classes for request bodies and responses (what goes over the wire).
router.py	The HTTP endpoints.
service.py	Business rules that are shared or complex (e.g. stage transitions, reports).

9. Configuration and environment variables

Secrets and settings are never written in code. They are read from **environment variables**, locally from a `.env` file (which Git ignores) and in production from Vercel's settings. `backend/app/core/config.py` reads them with **pydantic-settings**: each variable becomes a typed attribute, and the app refuses to start if a required one is missing.

backend/app/core/config.py (shortened)

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=ENV_FILE, extra="ignore")

    database_url: str                                  # required
    jwt_secret: str = Field(min_length=32)             # required, at least 32 chars
    access_token_expire_minutes: int = 480             # one working day
    frontend_base_url: str = "http://localhost:3000"
    cors_origins: str = "http://localhost:3000,..."
    cors_origin_regex: str | None = None
    google_auth_enabled: bool = False
    google_client_id: str = ""
    cloudinary_cloud_name: str = ""   # + api key / secret
    resend_api_key: str = ""
    email_from: str = "HireDesk <onboarding@resend.dev>"
    smtp_host: str = ""               # + port / username / password

    @field_validator("database_url")
    def use_psycopg_driver(cls, url):   # Neon gives postgres://; SQLAlchemy needs postgresql+psycopg://
        ...

@lru_cache
def get_settings() -> Settings:          # read once, reuse everywhere
    return Settings()
```

Variable	Used by	Purpose
DATABASE_URL	backend	Where PostgreSQL is. On Vercel it is injected by Neon (pooled).
DATABASE_URL_UNPOOLED	migrations	Direct (non-pooled) database connection, used for Alembic.
JWT_SECRET	backend	Secret key that signs login tokens. Anyone with it could forge logins.
ACCESS_TOKEN_EXPIRE_MINUTES	backend	Session length (default 480).
FRONTEND_BASE_URL	backend	Used to build links in emails (invites, dashboard).
CORS_ORIGINS / CORS_ORIGIN_REGEX	backend	Which websites may call the API from a browser.
GOOGLE_AUTH_ENABLED / GOOGLE_CLIENT_ID	backend	Turns on Google sign-in and says which Google client tokens must be for.
CLOUDINARY_*	backend	File storage credentials.
RESEND_API_KEY / EMAIL_FROM	backend	Email through Resend; sender name and address.
SMTP_HOST / PORT / USERNAME / PASSWORD	backend	Email through Gmail instead (used when SMTP_HOST is set).

Variable	Used by	Purpose
NEXT_PUBLIC_API_URL	frontend	Where the backend is. `NEXT_PUBLIC_` means it is baked into browser code (public).
NEXT_PUBLIC_GOOGLE_CLIENT_ID	frontend	The public Google client ID for the sign-in button.

> **Rule:** Anything starting with `NEXT_PUBLIC_` is visible to every visitor. Never put secrets there. The Google *client ID* is public by design; a client *secret* would not be.

10. Backend: how a request travels through FastAPI

When the browser calls, say, `POST /jobs`, this is what happens inside the backend:

1. **Vercel** receives the HTTPS request and runs the FastAPI app (`backend/app/main.py`) as a serverless function.
2. **CORS middleware** checks the request's *origin* (which website sent it). Only the HireDesk site (and localhost in development) is allowed.
3. FastAPI finds the matching **route** (`@router.post("")` in `jobs/router.py`).
4. FastAPI runs the route's **dependencies**: it opens a database session (`DbSession`), reads the token, loads the user (`CurrentUser`) and checks the role (`HiringManagerUser`).
5. FastAPI **validates the body** against a Pydantic schema (`JobCreate`). Bad input never reaches your code; the caller gets 422.
6. Your **function** runs: applies business rules, reads/writes the database, maybe queues an email.
7. The return value is **serialised** through the `response_model` (e.g. `JobRead`), so only declared fields are sent back. This is why `hashed_password` can never leak.
8. **Background tasks** (emails) run after the response is sent.

10.1 main.py – the entry point

backend/app/main.py (shortened)

```
app = FastAPI(title="HireDesk API", description="...", version="0.1.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=[...settings.cors_origins split by commas...],
    allow_origin_regex=get_settings().cors_origin_regex,   # Vercel preview URLs
    allow_credentials=True, allow_methods=["*"], allow_headers=["*"],
)

app.include_router(auth_router)        # /auth/...
app.include_router(users_router)       # /users/...
app.include_router(companies_router)   # /companies/me
app.include_router(jobs_router)        # /jobs/...
app.include_router(public_router)      # /public/jobs/...   (no login)
app.include_router(candidates_router)  # /applications/...
app.include_router(interviews_router)  # /interviews/...
app.include_router(scorecards_router)  # scorecards
app.include_router(reports_router)     # /reports/...

@app.get("/health")
def health(db: DbSession):
    db.exec(text("SELECT 1"))          # proves the database answers
    return {"status": "ok"}
```

Concept	Explanation
Middleware	Code that runs around *every* request, before and after the route. CORS is a middleware.
CORS	Cross-Origin Resource Sharing. Browsers block a page on one website from calling an API on another *unless* the API says that website is allowed. The site (hiredesk-app) and API (hiredesk-api) are different origins, so the API lists the site in `allow_origins`.

Concept	Explanation
Router	A group of endpoints with a shared URL prefix and tag (e.g. all `/jobs` endpoints). `include_router` plugs it into the app.
/docs	FastAPI automatically builds interactive documentation (Swagger UI) from the code. Open https://hiredesk-api.vercel.app/docs.

10.2 Dependency injection

A **dependency** is a function FastAPI runs *before* your route and passes the result in. You ask for it by type. This keeps login checks, database sessions and role checks out of every route body.

backend/app/jobs/router.py – the job's company and owner come from the logged-in user, never from the request

```
@router.post("", response_model=JobRead, status_code=201)
def create_job(body: JobCreate, manager: HiringManagerUser, db: DbSession):
    job = Job(company_id=manager.company_id, hiring_manager_id=manager.id, **body.model_dump())
    db.add(job)
    db.commit()
    return job
```

`body: JobCreate` means "validate the JSON body as JobCreate". `manager: HiringManagerUser` means "there must be a valid token for a user whose role is hiring_manager". `db: DbSession` means "give me a database session".

11. Backend: the database layer

11.1 The session

backend/app/db/session.py

```
engine = create_engine(
    get_settings().database_url,
    pool_pre_ping=True,                          # test a pooled connection before using it
    connect_args={"prepare_threshold": None},   # needed behind Neon's connection pooler
)

def get_db() -> Iterator[Session]:
    # One session per request. expire_on_commit=False keeps objects readable after commit().
    with Session(engine, expire_on_commit=False) as session:
        yield session

DbSession = Annotated[Session, Depends(get_db)]
```

Term	Meaning
Engine	The object that knows how to connect to the database and keeps a *pool* of open connections to reuse.
Session	A unit of work: you add/change objects, then `commit()` saves them all in one *transaction*.
Transaction	A group of changes that either all succeed or all fail (`rollback()`). Prevents half-saved data.
Connection pooler (PgBouncer)	Neon's `DATABASE_URL` goes through a pooler that shares a few real connections between many serverless requests.
prepare_threshold=None	psycopg normally caches *prepared statements* on the server after a few uses. A pooler in transaction mode can hand the next query to a different connection where that statement doesn't exist, causing errors. Turning it off avoids that.
`yield` in get_db	Code before `yield` runs before the request, code after (closing the session) runs after it. The `with` block guarantees the session closes.

11.2 Shared column helpers

backend/app/db/common.py

```
TIMESTAMPTZ = DateTime(timezone=True)       # every timestamp is time-zone aware

def utcnow() -> datetime:
    return datetime.now(timezone.utc)

def enum_column(enum_class):
    # Store enums as VARCHAR(20) text, not PostgreSQL ENUM types,
    # because PostgreSQL ENUMs are awkward to change later in migrations.
    return Enum(enum_class, native_enum=False, length=20,
                values_callable=lambda members: [m.value for m in members])
```

11.3 Models

A **model** is a Python class that maps to a table. SQLModel lets one class be both the table definition and a validated data object.

backend/app/users/models.py

```
class Role(StrEnum):
    company_admin = "company_admin"
    hiring_manager = "hiring_manager"
    interviewer = "interviewer"


class User(SQLModel, table=True):
    __tablename__ = "users"
    id: uuid.UUID = Field(default_factory=uuid.uuid4, primary_key=True)
    company_id: uuid.UUID = Field(foreign_key="companies.id")
    email: str = Field(unique=True, index=True)   # the unique index also makes login lookup fast
    full_name: str
    hashed_password: str
    password_login_enabled: bool = Field(default=True, nullable=False)
    role: Role = Field(sa_type=enum_column(Role))
```

- `table=True` makes it a real table; without it, it is only a data shape.
- `default_factory=uuid.uuid4` generates a new UUID for each row.
- `foreign_key="companies.id"` creates the FK relationship from the ERD.
- `index=True` creates a database index, a lookup structure that makes searching by that column fast.
- `StrEnum` makes the enum values plain strings, so they serialise to JSON naturally.

The other models follow the same pattern: `Company`, `Invitation`, `Job` (with `JobStatus` and `Stage` enums), `Application`, `StageHistory`, `Interview` (with `InterviewStatus` and `MeetingType`), `Scorecard` and `ScorecardRating` (with `ScorecardStatus` and `Recommendation`).

11.4 Schemas vs models

Models describe what is *stored*. **Schemas** describe what goes *over the wire*. They are kept separate on purpose:

backend/app/users/schemas.py

```
class UserRead(BaseModel):                  # what the API returns about a user
    model_config = ConfigDict(from_attributes=True)   # can be built from a User object
    id: uuid.UUID
    company_id: uuid.UUID
    email: str
    full_name: str
    role: Role
    password_login_enabled: bool
    # no hashed_password field -> it can never be sent to a browser
```

Schemas also hold validation: `RegisterRequest.password: str = Field(min_length=8, max_length=128)`, `LowerEmail` lower-cases emails, `JobCreate` rejects `salary_min > salary_max`, `InterviewSchedule` rejects past times and requires a link for virtual meetings.

12. Backend: security – passwords, tokens and permissions

12.1 Password hashing

Passwords are never stored. A **hash** is a one-way scramble: you can check a password against it, but you cannot turn it back into the password. **Argon2** is deliberately slow and memory-hungry, so attackers who steal the database cannot guess passwords quickly.

backend/app/core/security.py

```
password_hash = PasswordHash.recommended()     # Argon2


def hash_password(password: str) -> str:
    return password_hash.hash(password)


def verify_password(password: str, hashed_password: str) -> bool:
    return password_hash.verify(password, hashed_password)
```

12.2 JWT login tokens

After login, the server gives the browser a **JWT (JSON Web Token)**: a small piece of text with three parts – header, payload and signature. The payload says *who* (`sub` = the user id) and *until when* (`exp`). The signature is made with `JWT_SECRET`; if anyone changes the payload, the signature no longer matches and the token is rejected.

backend/app/core/security.py

```
ALGORITHM = "HS256"


def create_access_token(user_id) -> str:
    expires_at = datetime.now(timezone.utc) + timedelta(minutes=settings.access_token_expire_minutes)
    return jwt.encode({"sub": str(user_id), "exp": expires_at}, settings.jwt_secret, algorithm=ALGORITHM)


def decode_access_token(token: str) -> uuid.UUID | None:
    try:
        payload = jwt.decode(token, get_settings().jwt_secret, algorithms=[ALGORITHM])
        return uuid.UUID(payload["sub"])
    except (jwt.PyJWTError, KeyError, ValueError):
        return None                        # bad, tampered or expired
```

The browser sends it on every private request as a header: `Authorization: Bearer <token>`. This is called **bearer authentication**: whoever *bears* the token is treated as that user, which is why it must be kept private.

12.3 Authentication and role dependencies

backend/app/auth/dependencies.py

```
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="auth/login")   # reads the Bearer header


def get_current_user(token = Depends(oauth2_scheme), db: DbSession) -> User:
    user_id = decode_access_token(token)
    user = db.get(User, user_id) if user_id else None    # loaded fresh on every request
    if user is None:
        raise HTTPException(401, "Invalid or expired token")
    return user
```

```
CurrentUser = Annotated[User, Depends(get_current_user)]

def require_roles(*allowed_roles):
    def check_role(user: CurrentUser) -> User:
        if user.role not in allowed_roles:
            raise HTTPException(403, "You do not have permission to do this")
        return user
    return check_role

AdminUser         = Annotated[User, Depends(require_roles(Role.company_admin))]
ManagerUser       = Annotated[User, Depends(require_roles(Role.company_admin, Role.hiring_manager))]
HiringManagerUser = Annotated[User, Depends(require_roles(Role.hiring_manager))]
InterviewerUser   = Annotated[User, Depends(require_roles(Role.interviewer))]
```

401 vs 403: **401 Unauthorized** = "I don't know who you are" (no/bad token). **403 Forbidden** = "I know who you are, but your role can't do this."

12.4 Permissions: who can see which rows

Roles decide which *actions* you may take. But you also need to limit which *rows* you see: a hiring manager must only see their own jobs, and nobody may see another company. All of these rules live in one file, `backend/app/auth/permissions.py`, and every list and detail endpoint goes through it.

backend/app/auth/permissions.py (shortened)

```
def _limit_to_visible_jobs(query, user):
    query = query.where(Job.company_id == user.company_id)          # never another company
    if user.role == Role.hiring_manager:
        query = query.where(Job.hiring_manager_id == user.id)       # only their own jobs
    return query

def visible_applications(user):
    query = select(Application, Job).join(Job, Job.id == Application.job_id)
    query = _limit_to_visible_jobs(query, user)
    if user.role == Role.company_admin:
        query = query.where(false())            # admins don't browse CVs
    elif user.role == Role.interviewer:
        assigned = select(Interview.application_id).where(
            Interview.interviewer_id == user.id, Interview.status != InterviewStatus.cancelled)
        query = query.where(Application.id.in_(assigned))   # only candidates assigned to them
    return query

def get_visible_application(db, user, application_id):
    row = db.exec(visible_applications(user).where(Application.id == application_id)).first()
    if row is None:
        raise HTTPException(404, "Application not found")   # 404, not 403: don't reveal it exists
    return row
```

Interview answer: "Authorization is enforced on the server in one module. Every query starts from the user's *visible* set, so a hiring manager physically cannot load another manager's candidate. Anything out of scope returns 404 so its existence isn't revealed."

13. Backend: sign-up, login, Google and invitations (auth/router.py)

13.1 Register and login

- `POST /auth/register` creates a `Company`, then a `User` with role `company_admin` and a hashed password. `db.flush()` inserts the company first so its id exists for the user's foreign key. A duplicate email hits the unique index and becomes a 409.
- `POST /auth/login` uses the standard OAuth2 *form* (fields `username` and `password`; the email goes in `username`). It looks up the user by lower-cased email, verifies the hash, and returns `{access_token, token_type: "bearer"}`. A wrong email or password gives the *same* message, so attackers can't tell which emails exist.
- `GET /auth/me` returns the current user (via `CurrentUser`).

13.2 Google sign-in

The browser shows Google's button. When the user picks an account, Google gives the browser an **ID token**: a JWT signed by Google containing the email and whether it is verified. The browser sends it to `POST /auth/google`. The server must not trust the browser, so it verifies the token itself:

backend/app/auth/router.py – google_auth (shortened)

```
claims = google_id_token.verify_oauth2_token(
    body.id_token, google_requests.Request(), settings.google_client_id)
#   ^ checks Google's signature, the expiry, and that the token was issued FOR our client id
if claims.get("email_verified") is not True or not claims.get("email"):
    raise HTTPException(401, "Google email is not verified")
email = claims["email"].lower()


# 1. Invite page: the Google email must match the invited email.
# 2. Existing user  -> sign in.
# 3. Open invitation for this email -> create the user in the inviting company with the invited role.
# 4. Otherwise, only the Register page (create_company=true) may create a new company;
#    the login page gets a clear 404 "No HireDesk account uses this Google email".
```

Google-created accounts get a random password they never know, and `password_login_enabled = False`. They can set a real password later on the Account page.

13.3 Changing a password

backend/app/auth/router.py

```
@router.put("/password", response_model=UserRead)
def change_password(body: ChangePasswordRequest, user: CurrentUser, db: DbSession, background_tasks):
    if user.password_login_enabled:
        # 400, not 401: a wrong current password must not look like an expired session
        if not body.current_password or not verify_password(body.current_password, user.hashed_password):
            raise HTTPException(400, "Your current password is incorrect")
        if body.current_password == body.password:
            raise HTTPException(400, "Choose a new password that is different from the current one")
    user.hashed_password = hash_password(body.password)
    user.password_login_enabled = True
    ...commit, then email "Your HireDesk password was changed"...
```

13.4 Accepting an invitation

- The admin's invite creates an `Invitation` and a random token (`secrets.token_urlsafe(32)`). Only `sha256(token)` is stored. The link is `/accept-invite?token=<token>`.
- `POST /auth/accept-invite` hashes the token it receives and looks for an invitation with that hash that is **not accepted and not expired** (`_find_open_invitation`). If none: 410 Gone.
- It creates the user with the invitation's company, name and role, sets `accepted_at`, and emails the admins "X joined your team". A second use of the same link fails because `accepted_at` is now set.

> **Why hash the token?** If someone could read the database, plain tokens would let them join any company. A SHA-256 hash can be checked against but not reversed, exactly like passwords.

14. Backend: the team and invitations (users/router.py)

Endpoint	Who	What it does
POST /users	Admin	Invite someone. Rejects emails already registered, or with an open invitation ("resend it instead"). Reuses an *expired* invitation row instead of creating a duplicate. Sends the email immediately and returns `email_sent` and the link.
GET /users/invitations	Admin	Pending (not accepted) invitations for the admin's company, newest first. Never returns tokens.
POST /users/invitations/{id}/resend	Admin	New token (old link stops working), 7 more days, emails again.
DELETE /users/invitations/{id}	Admin	Revoke: deletes the invitation so its link no longer works.
GET /users	Admin, hiring manager	Team members of the company, optionally filtered by role (used to pick an interviewer).

The shared helper `_issue_and_send` creates a fresh token, stores its hash and new expiry, commits, builds the link from `FRONTEND_BASE_URL`, renders the email and sends it:

backend/app/users/router.py

```
def _issue_and_send(db, invitation, admin) -> UserInvitationRead:
    token = secrets.token_urlsafe(32)
    invitation.token_hash = hashlib.sha256(token.encode()).hexdigest()
    invitation.expires_at = utcnow() + timedelta(days=INVITATION_DAYS)
    db.add(invitation); db.commit()
    invite_url = f"{settings.frontend_base_url}/accept-invite?token={token}"
    subject, text, html = invitation_email(...)
    # Sent now, not in the background, so the admin learns whether it went out
    email_sent = send_email(to=invitation.email, subject=subject, text=text, html=html)
    return UserInvitationRead(..., invite_url=invite_url, email_sent=email_sent)
```

`invitation_email()` returns a plain-text version and an HTML version with a "Join the team" button. Names are passed through `html.escape()` so a name like `<script>` is shown as text instead of being run: this prevents **HTML injection**.

15. Backend: jobs, public applications and file uploads

15.1 Jobs

- `POST /jobs` (hiring manager): new jobs start as **draft** and belong to the creator.
- `GET /jobs`, `GET /jobs/{id}` (admin or hiring manager): through `visible_jobs`, so managers see only their own.
- `PATCH /jobs/{id}` (hiring manager): partial update. `model_dump(exclude_unset=True)` changes only the fields sent. The salary range is re-checked against the stored values.

15.2 The public side (no login)

- `GET /public/jobs/{id}`: only **open** jobs are visible; anything else is 404 “This job is not accepting applications”.
- `POST /public/jobs/{id}/uploads`: signs a direct upload (below).
- `POST /public/jobs/{id}/applications`: the application form. It normalises the name and phone (collapses line breaks so they can’t break email subjects), checks for a duplicate email, gets the CV and documents, inserts the `Application` and the first `StageHistory` row (from_stage = NULL), commits, and emails the candidate a confirmation.

15.3 Uploading large files on a serverless host

The problem: Vercel functions reject request bodies over **4.5 MB**, but CVs may be 5 MB and documents 10 MB. **The solution:** the browser uploads straight to Cloudinary with a one-time *signed* permission, and the server checks the file afterwards.

1. Browser → `POST /public/jobs/{id}/uploads` with `{kind, filename, size}`. The server checks type and claimed size, invents a random public id like `hiredesk/cvs/<32 hex>.pdf`, and signs `{public_id, timestamp}` with the Cloudinary secret (`cloudinary.utils.api_sign_request`). The secret itself never leaves the server.
2. Browser → Cloudinary: POSTs the file with the signed fields. Cloudinary accepts it only because the signature matches *that exact* public id.
3. Browser → `POST /applications` with `cv_public_id` (and `document_public_ids` + `document_names`).
4. Server → `confirm_upload`: the id must match our folder and pattern (regex), the extension must be allowed, then it asks Cloudinary for the **real** size (`cloudinary.api.resource`). Too big → the file is deleted and the request gets 413.

backend/app/core/storage.py

```
def confirm_upload(kind, public_id) -> str:
    folder, extensions, max_bytes, size_error, type_error = RULES[kind]
    match = re.fullmatch(rf"{re.escape(folder)}/[0-9a-f]{{32}}(\.[a-z]+)", public_id)
    if match is None:
        raise HTTPException(422, "The uploaded file reference is not valid")
    ...
    # Ask Cloudinary for the real size: never trust the size the browser claimed.
    resource = cloudinary.api.resource(public_id, resource_type="raw")
    if int(resource.get("bytes", 0)) > max_bytes:
        cloudinary.uploader.destroy(public_id, resource_type="raw")
        raise HTTPException(413, size_error)
```

```
    return resource["secure_url"]
```

16. Backend: the hiring pipeline (candidates)

16.1 The state machine

A **state machine** is a set of states and the allowed moves between them. The rules live in one dictionary:

backend/app/candidates/service.py

```
ALLOWED_TRANSITIONS = {
    Stage.applied:   {Stage.screen, Stage.rejected},
    Stage.screen:    {Stage.interview, Stage.rejected},
    Stage.interview: {Stage.offer, Stage.rejected},
    Stage.offer:     {Stage.hired, Stage.rejected},
    Stage.hired:     set(),     # final
    Stage.rejected:  set(),     # final
}
```

16.2 Changing a stage safely (row locking)

The problem: two managers click “Move to Interview” at the same moment. Both read stage = screen, both decide the move is valid, and history gets two rows claiming the same “from”. **The solution:** lock the row.

backend/app/candidates/service.py

```
def change_stage(db, application_id, new_stage, user) -> Application:
    application = db.exec(
        select(Application)
        .where(Application.id == application_id)
        .with_for_update()                                  # SELECT ... FOR UPDATE: lock this row
        .execution_options(populate_existing=True)          # re-read the CURRENT stage
    ).one()
    if new_stage not in ALLOWED_TRANSITIONS[application.stage]:
        raise HTTPException(
            409, f"A candidate in '{application.stage}' cannot be moved to '{new_stage}'")
    db.add(StageHistory(application_id=application.id, from_stage=application.stage,
                        to_stage=new_stage, changed_by_id=user.id))
    application.stage = new_stage
    db.commit()                                             # the lock is released here
    return application
```

`FOR UPDATE` makes the second request **wait** until the first commits. It then sees the new stage, and its move is checked against that. A test (`test_simultaneous_stage_changes_keep_the_history_consistent`) fires requests in parallel threads to prove this.

16.3 The candidate endpoints

Endpoint	Who	Notes
GET /applications	Anyone signed in (scoped)	Filters by job and stage; sorts by date, match score, salary or interviewer recommendation; pagination (limit 1–200, offset).
GET /applications/{id}	Scoped	404 if not visible.
PATCH /applications/{id}/stage	Hiring manager	Moves forward (screen/interview/offer). Emails

Endpoint	Who	Notes
		the candidate and the admins.
PATCH /applications/{id}/assessment	Hiring manager	Match score 0–100 and notes. Emails admins a summary.
POST /applications/{id}/decision	Hiring manager	`hired` (only from offer) or `rejected`. Emails the candidate and admins.
GET /applications/{id}/history	Admin or hiring manager (scoped)	The audit trail of moves.

Sorting by interviewer recommendation uses a SQL `CASE` expression to rank recommend = 2, maybe = 1, not recommend = 0, and `nullslast()` keeps candidates without a value at the bottom.

17. Backend: interviews and the no-double-booking rule

17.1 Two steps: assign, then schedule

- `POST /interviews` (hiring manager): assign an interviewer to a candidate. Checks the candidate isn't hired/rejected and the interviewer is an *interviewer* in the *same company*. Creates the `Interview` (status `assigned`, no time yet) **and** an empty pending `Scorecard`. Emails the interviewer and admins.
- `PATCH /interviews/{id}/schedule` (that interviewer only): sets a future start, a duration of 15–240 minutes, and either a meeting URL (virtual) or a location (in person). Status becomes `scheduled`. Emails the candidate and admins.
- `POST /interviews/{id}/cancel` (hiring manager): status `cancelled`, which frees the time slot.

17.2 The exclusion constraint

The problem: an interviewer must never have two overlapping interviews. Checking in Python ("is there already one at that time?") has a race: two requests can both check, both find nothing, and both save. **The solution:** let PostgreSQL enforce it.

backend/alembic/versions/0002_hiring_workflows.py

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

ALTER TABLE interviews
ADD CONSTRAINT no_overlapping_interviews
EXCLUDE USING gist (
    interviewer_id WITH =,                    -- same interviewer, AND
    tstzrange(starts_at, ends_at) WITH &&     -- time ranges overlap  ->  not allowed
)
WHERE (status = 'scheduled');                 -- assigned/cancelled/completed rows don't block
```

Piece	Meaning
EXCLUDE constraint	Like UNIQUE, but more general: no two rows may satisfy *all* the listed comparisons at once.
`tstzrange(start, end)`	A time-range value. `[start, end)` includes the start but not the end, so back-to-back interviews (10:00–11:00, 11:00–12:00) are fine.
`&&`	The "overlaps" operator for ranges.
GiST	An index type that can handle ranges and overlap searches.
btree_gist	A PostgreSQL extension that lets a GiST index also compare plain values like a UUID with `=`.

backend/app/interviews/router.py – turning the database error into a friendly 409

```
try:
    db.add(interview)
    db.commit()
except IntegrityError as error:
    db.rollback()
    if isinstance(error.orig, ExclusionViolation):        # psycopg's specific error type
        raise HTTPException(409, "You already have an interview at that time") from error
```

```
    raise
```

18. Backend: scorecards and fair feedback

18.1 Submitting a scorecard

backend/app/scorecards/router.py (shortened)

```
row = db.exec(
    select(Scorecard, Interview).join(Interview, ...)
    .where(Interview.id == interview_id,
           Interview.interviewer_id == interviewer.id,          # only your own interview
           Interview.status != InterviewStatus.cancelled)
    .with_for_update()                       # lock: a double click can't submit twice
    .execution_options(populate_existing=True)
).first()
if row is None:
    raise HTTPException(404, "Interview not found")
if scorecard.status == ScorecardStatus.submitted:
    raise HTTPException(409, "already submitted")
if interview.status != InterviewStatus.scheduled:
    raise HTTPException(409, "must be scheduled")
if interview.starts_at is None or interview.starts_at > utcnow():
    raise HTTPException(409, "You can submit the scorecard once the interview has started")
...save ratings, status = submitted, interview.status = completed...
```

Then the candidate's overall `interviewer_recommendation` is recomputed from all submitted scorecards: **any** "not recommend" → not recommend; **all** "recommend" → recommend; otherwise maybe. Admins are emailed the ratings and recommendation.

18.2 The privacy rule (avoiding anchoring bias)

Anchoring bias: people unconsciously copy the first opinion they see. If interviewer B could read A's scorecard first, B's feedback would no longer be independent. So:

backend/app/scorecards/router.py

```
def hide_scorecards_from_interviewer(rows, interviewer):
    own_rows = [(card, iv) for card, iv in rows if iv.interviewer_id == interviewer.id]
    has_submitted_all_own = all(card.status == ScorecardStatus.submitted for card, _ in own_rows)
    if not has_submitted_all_own:
        return own_rows                                  # only your own until you've submitted
    return [(card, iv) for card, iv in rows
            if iv.interviewer_id == interviewer.id or card.status == ScorecardStatus.submitted]
```

Managers see every scorecard and an **aggregate score**: the average of each submitted scorecard's average rating. Interviewers never receive the aggregate.

19. Backend: email, notifications and reports

19.1 Sending email

backend/app/core/email.py (shortened)

```
def send_email(to, subject, text, html=None) -> bool:
    try:
        if settings.smtp_host:            _send_smtp(to, subject, text, html)        # e.g. Gmail
        elif settings.resend_api_key:     resend.Emails.send({...})
        else:
            logger.warning("No email provider is configured"); return False
    except Exception:
        logger.exception("Could not send the email to %s", to); return False
    return True
```

- It **returns True/False instead of raising**, so a broken email provider can never fail a hire or an application.
- **SMTP** uses STARTTLS (encryption) on port 587, logs in with the Gmail app password, and sends a message with both a plain-text and an HTML part. Gmail only sends *as* the logged-in account, so that address becomes the From address.
- **Resend's test sender** (`onboarding@resend.dev`) only delivers to the Resend account owner. That is why invitations need a verified domain or Gmail.

19.2 Background tasks

`background_tasks.add_task(send_email, ...)` tells FastAPI to send the email *after* the response is returned, so the user doesn't wait for the email server. Invitations are the exception: they are sent immediately so the admin can be told whether delivery worked.

19.3 Admin notifications

`core/notifications.py` has `notify_company_admins(db, background_tasks, company_id, subject, text)`, which emails every admin of the company with a "[HireDesk]" subject prefix and a link to the dashboard. It is called on stage moves, assessments, hires, rejections, interviews assigned/scheduled/cancelled, scorecards submitted and teammates joining.

19.4 The progress report

- `reports/service.py` → `build_progress_report(db, company_id, period_days)` loads the company's users, jobs, applications, interviews, scorecards and recent history, then counts in Python: totals, the pipeline per stage, per-job progress, each teammate's workload (including overdue scorecards), recent activity, results and scorecards.
- `GET /reports/progress?days=7|30|90` (admin) feeds the admin dashboard; `POST /reports/progress/email` emails it to the admin.
- `render_report_text()` formats the same data as a plain-text email.
- `python -m app.reports.send_progress_reports` emails every company's admins. GitHub Actions runs it every Monday at 08:00 UTC.

20. Backend: migrations and tests

20.1 Alembic migrations

A **migration** is a versioned script that changes the database structure (add a table, add a column…). Each has an `upgrade()` and a `downgrade()` and points to the previous one (`down_revision`), forming a chain. `alembic upgrade head` applies everything not yet applied; Alembic remembers the current version in a table called `alembic_version`.

Migration	What it does
0001 initial schema	Creates companies, users, jobs, applications, stage_history, interviews, scorecards, scorecard_ratings; enables btree_gist; adds the first no-overlap constraint.
0002 hiring workflows	Adds job requirements and salary range; candidate qualifications, expected salary, match score, notes, recommendation, documents; interviews become two-step (`assigned` with no time); meeting type/URL/location; re-creates the overlap constraint for `scheduled` only.
0003 team invitations	Creates the `invitations` table and its indexes.
0004 password login	Adds `users.password_login_enabled` (true for existing users).
0005 drop duplicate constraint	Removes a redundant second unique rule on `invitations.token_hash`, found by `alembic check`.

`alembic check` compares the models with the real database and fails if a model was changed without a migration. CI now runs it on every push.

20.2 The tests

Automated tests are code that checks other code. `pytest` finds functions named `test_*`, runs them, and reports failures. HireDesk has **73** backend tests that run against a real PostgreSQL test database.

Piece	What it does
conftest.py	Shared setup. Points the app at `hiredesk_test` (refuses any database name not ending in `_test`), runs the real migrations once, empties every table before each test, and fakes Cloudinary.
`client` fixture	A FastAPI `TestClient` that calls the API in memory, no server needed.
`World` / `world` fixture	Builds a realistic company: an admin, a hiring manager, two interviewers (Ann and Bob) and one open job. Helpers: `apply()`, `schedule()`, `slot()`, `hold()` (moves an interview into the past), `parallel()` (fires requests at the same instant from threads).
`outbox` fixture	Replaces `send_email` in every module with a fake that records messages, so tests can check emails were sent without sending any.
test_auth.py	Register, login, Google (mocked), invites, roles, company isolation.
test_candidates.py	Applying, duplicates, stage rules, decisions, assessment emails, sorting, manager

Piece	What it does
	scope, simultaneous moves.
test_interviews.py	Assigning, scheduling, overlaps, cancellation, permissions.
test_scorecards.py	Submitting, validation, double submit, privacy rule, aggregate, timing.
test_uploads.py	Signed uploads, real-size checks, invalid references.
test_invitation_email.py	Invite emails, failure reporting, resend/revoke, SMTP.
test_progress.py	Password change, admin notifications, reports.

A **fixture** is a reusable piece of test setup that pytest passes into a test by its parameter name. **Mocking** means replacing a real dependency (Google, Cloudinary, email) with a fake that behaves predictably, so tests are fast and don’t touch the outside world.

21. Frontend: Next.js basics you need

21.1 Components and JSX

A **React component** is a function that returns what to show. The HTML-like syntax inside it is **JSX**. Components can take inputs (**props**) and remember values (**state**).

frontend/src/components/shared/stage-badge.tsx

```
export function StageBadge({ stage }: { stage: Stage }) {      // props: { stage }
  return (
    <Badge className="bg-card ring-1 ring-border">
      <span className="size-2 rounded-full" style={{ background: STAGE_COLOR[stage] }} />
      {STAGE_LABEL[stage]}                                      {/* curly braces = JavaScript */}
    </Badge>
  );
}
```

Hook	What it does
`useState`	Remembers a value between renders; changing it re-draws the component. E.g. whether a dialog is open.
`useEffect`	Runs code *after* rendering, e.g. loading a script or subscribing to a store. Returns a cleanup function.
`useRef`	Holds a value or a DOM element without causing re-renders (used by the Google button).
Custom hooks	Functions starting with `use` that bundle logic, e.g. `useJobs()` in `lib/queries.ts`.

21.2 The App Router: folders are URLs

In Next.js's **App Router**, every folder under `src/app` with a `page.tsx` becomes a URL:

File	URL	Who sees it
app/page.tsx	/	Everyone: the landing page
app/login/page.tsx	/login	Public
app/register/page.tsx	/register	Public: create a company
app/accept-invite/page.tsx	/accept-invite?token=…	Invited teammates
app/apply/[jobId]/page.tsx	/apply/123…	Candidates. `[jobId]` is a *dynamic segment*: any value.
app/(app)/overview/page.tsx	/overview	Signed in: the dashboard
app/(app)/jobs/…, candidates/…, interviews/, team/, account/	/jobs, /candidates, …	Signed in, by role
app/not-found.tsx, app/error.tsx	any missing page / crash	Everyone
app/icon.svg	/icon.svg	The favicon

- **`layout.tsx`** wraps every page below it. `app/layout.tsx` loads the fonts and `<Providers>`; `app/(app)/layout.tsx` adds the login check and the sidebar.
- **`(app)`** in brackets is a *route group*: it groups pages under one layout without adding `/app` to the URL.
- **`"use client"`** at the top of a file means it runs in the browser (it uses state, effects or browser APIs). Without it, a component renders on the server.

22. Frontend: talking to the backend

22.1 The API client (lib/api.ts)

All network calls go through one small wrapper around the browser's `fetch`. It adds the token, sets JSON headers, turns FastAPI's error format into a readable message, and handles expired sessions.

frontend/src/lib/api.ts (shortened)

```
async function request<T>(path: string, init: RequestInit = {}, auth = true): Promise<T> {
  const headers = new Headers(init.headers);
  const token = useAuthStore.getState().token;
  const sendsToken = auth && !!token;
  if (sendsToken) headers.set("Authorization", `Bearer ${token}`);
  if (typeof init.body === "string") headers.set("Content-Type", "application/json");

  let response: Response;
  try {
    response = await fetch(`${API_URL}${path}`, { ...init, headers });
  } catch {
    throw new ApiError("Cannot reach the server. Check that the backend is running.", 0);
  }

  // Only an expired session sends the user back to /login, and to the same page afterwards.
  if (response.status === 401 && sendsToken) {
    useAuthStore.getState().logout();
    const next = encodeURIComponent(location.pathname + location.search);
    window.location.href = `/login?expired=1&next=${next}`;
  }
  const body = response.status === 204 ? null : await response.json().catch(() => null);
  if (!response.ok) throw new ApiError(errorMessage(body, "Something went wrong"), response.status);
  return body as T;
}

export const api = { get, post, patch, put, del, postForm };   // thin shortcuts
```

`<T>` is a **generic**: the caller says what type comes back, e.g. `api.get<Job[]>("/jobs")`, so TypeScript knows the result is a list of jobs.

22.2 The auth store (stores/auth-store.ts)

frontend/src/stores/auth-store.ts

```
export const useAuthStore = create<AuthState>()(
  persist(
    (set) => ({
      token: null, user: null,
      setSession: (token, user) => set({ token, user }),
      logout: () => set({ token: null, user: null }),
    }),
    { name: "hiredesk-auth" },        // saved in the browser's localStorage
  ),
);
```

Zustand keeps the token and user in memory, and the `persist` middleware saves them in **localStorage** so you stay signed in after a refresh. The protected layout waits for this to load ("hydrate") before deciding whether to redirect to `/login`.

22.3 Server data with TanStack Query (lib/queries.ts)

frontend/src/lib/queries.ts

```
export const useJobs = (enabled = true) =>
  useQuery({ queryKey: ["jobs"], queryFn: () => api.get<Job[]>("/jobs"), enabled });

export function useCreateJob() {
  const refresh = useRefresh();
  return useMutation({
    mutationFn: (body) => api.post<Job>("/jobs", body),
    onSuccess: () => refresh("jobs"),           // mark "jobs" stale -> lists refetch automatically
  });
}
```

Idea	Meaning
Query	Reading data. `useQuery` returns `data`, `isLoading`, `isError`, `error`.
Query key	The cache name, e.g. `["applications", jobId, sortBy]`. Same key = shared cached data.
Mutation	Changing data (`useMutation`), with `onSuccess`/`onError` callbacks.
Invalidation	After a change, mark related keys stale so screens refetch fresh data.
Optimistic update	`useMoveStage` updates the board *before* the server answers, and rolls back if the server refuses (`onMutate` / `onError`).
placeholderData: keepPreviousData	The admin report keeps showing the old period while a new one loads, so the page doesn't flash.

22.4 Clearing data on logout (privacy fix)

frontend/src/components/providers.tsx

```
useEffect(() =>
  useAuthStore.subscribe((state, previous) => {
    if (state.user?.id !== previous.user?.id || (previous.token && !state.token)) queryClient.clear();
  }), [queryClient]);
```

Before this fix, the cache survived logout, so the next person on the same browser could briefly see the previous user's candidates. Now the cache is wiped whenever the account changes.

23. Frontend: types, validation and forms

- **`lib/types.ts`** mirrors the backend's response schemas in TypeScript (`User`, `Job`, `Application`, `Interview`, `Scorecard`, `ProgressReport`, …). If the backend adds a field, add it here too.
- **`lib/schemas.ts`** holds **Zod** schemas for forms, e.g. a password of at least 8 characters, matching confirmation, a valid email, a meeting link required for virtual interviews, and a start time in the future.
- **`lib/constants.ts` / `lib/stages.ts`** hold labels, colours and where each role lands after login (`HOME`, now `/overview` for everyone).

Zod + React Hook Form working together

```
export const changePasswordSchema = z.object({
  current_password: z.string().max(128).optional(),
  password: z.string().min(8, "Use at least 8 characters").max(128),
  confirm_password: z.string(),
}).refine((v) => v.password === v.confirm_password, {
  message: "Passwords do not match", path: ["confirm_password"],
});

// In the component:
const { register, handleSubmit, formState: { errors } } =
  useForm<PasswordValues>({ resolver: zodResolver(changePasswordSchema) });
<Input {...register("password")} />           // connects the input to the form
{errors.password?.message}                      // shows the Zod message
```

> **Remember:** Frontend validation is for *user experience* (instant feedback). The backend validates again with Pydantic, because anyone can call the API directly. Never rely on the frontend for security.

24. Frontend: the public pages

24.1 Landing page (app/page.tsx)

The marketing page: a hero image with “Create your workspace”, the workflow, role cards and contact details, with `ScrollReveal` animations that respect “reduce motion” settings.

24.2 Login, register and Google sign-in

- **Login** posts the OAuth2 form to `/auth/login`, saves the token, fetches `/auth/me`, then goes to `next` (the page you were on) or your dashboard. `safeNext()` only accepts paths starting with a single `/`, so a crafted link can’t send people to another website (an *open redirect*).
- **Register** creates the company, then logs in with the same details. Its Google button sends `create_company: true`.
- **GoogleSignInButton** loads `https://accounts.google.com/gsi/client` once, calls `google.accounts.id.initialize({client_id, callback})` and renders Google’s button. The callback receives `response.credential` (the ID token) and passes it up.

> **Google’s “Access blocked” error:** Google only shows its button on websites listed as **Authorized JavaScript origins** for the client ID. The live site had to be added in Google Cloud Console; nothing in the code could fix it.

24.3 Accepting an invitation (app/accept-invite)

Reads `token` from the URL. Either choose a password (`POST /auth/accept-invite`, then log in) or continue with Google (`POST /auth/google` with the `invite_token`, which must match the invited email).

24.4 Applying for a job (app/apply/[jobId])

frontend/src/app/apply/[jobId]/page.tsx

```
setPhase("uploading");
const [cvId, ...documentIds] = await Promise.all([          // upload all files at the same time
  uploadDirect(jobId, "cv", cv),
  ...documents.map((document) => uploadDirect(jobId, "document", document)),
]);
setPhase("sending");
const form = new FormData();
form.append("full_name", values.full_name);  ...
form.append("cv_public_id", cvId);
documentIds.forEach((id, i) => {
  form.append("document_public_ids", id);
  form.append("document_names", documents[i].name);
});
await api.postForm(`/public/jobs/${jobId}/applications`, form);
```

`uploadDirect` (`lib/uploads.ts`) asks the API for a signature, then POSTs the file plus the signed fields straight to Cloudinary, and returns the public id (Section 15.3).

25. Frontend: the signed-in app

25.1 Protected layout and app shell

- `app/(app)/layout.tsx` waits until the browser has mounted and the auth store has loaded. With no token it redirects to `/login?next=…`; otherwise it renders `AppShell`.
- `AppShell` (`components/layout/app-shell.tsx`) is the sidebar on desktop and a menu dialog on mobile. `useNavItems()` builds the menu **by role**: everyone gets Dashboard and Account; hiring managers get Jobs, Candidates, Interviews; interviewers get Assigned candidates and Interviews; admins get Team.

> **Important:** Hiding menu items is *convenience*, not security. If an interviewer typed `/jobs`, the API would still answer 403. The server is always the gatekeeper.

25.2 Pages

Page	What it shows and does
/overview	Picks `AdminDashboard`, `ManagerDashboard` or `InterviewerDashboard` from the user's role (Section 26).
/jobs	The hiring manager's jobs; "New job" opens `JobFormDialog`.
/jobs/[jobId]	A **board** (Kanban) with one column per stage and candidate cards; "Copy apply link"; edit; details tab.
/candidates	Search, filter by stage, sort (managers); each row links to the candidate.
/candidates/[applicationId]	Profile, CV and documents, role requirements, the assessment form, activity history, interviews (assign, cancel, "Fill scorecard" once started), scorecards with the aggregate, and the stage menu (move / hire / reject with confirmation).
/interviews	Tabs: awaiting schedule, scheduled, completed, cancelled. Interviewers can Schedule (dialog with a future-only date picker) and Fill scorecard.
/team	Admin: members with roles; "Add team member" (shows whether the invite was emailed, plus a copyable link); **Pending invitations** with Resend and Revoke.
/account	Profile and Change password (or Set a password for Google-only accounts).

25.3 Reusable components

Folder	Examples	Purpose
components/ui	button, card, dialog, dropdown-menu, input, select, tabs, badge, avatar, skeleton	Generic, accessible building blocks (shadcn/ui on Radix).
components/shared	stage-thread, stage-badge, stage-menu, empty-state, field, page-header, confirm-dialog, google-sign-in-button	HireDesk-specific pieces reused across pages.
components/features	job-form-dialog, schedule-dialog, interview-	Bigger features with their own

Folder	Examples	Purpose
	schedule-dialog, scorecard-dialog, cv-dropzone, candidate-card, dashboard/*	data and logic.
components/layout	app-shell, auth-layout, logo	Page frames.

The **stage thread** is HireDesk's signature visual: a line through Applied → Screen → Interview → Offer that fills up to the candidate's current stage, with a tick on completed stages.

26. Frontend: the dashboards

Everyone lands on `/overview` and sees a dashboard about *their own* work. Shared pieces live in `components/features/dashboard/parts.tsx`: `DashboardHero` (time-aware greeting, one summary sentence, one big number), `StatTile`, `Section`, `TaskRow`, `StageBars`, `OutcomeTile`, `Agenda` (interviews grouped by Today/Tomorrow/date) and `Timeline`.

Dashboard	Data source	Shows
Interviewer	`/interviews` + `/applications` (already limited to them)	Interviews waiting for a date, scorecards due (link to the candidate page), upcoming schedule with Join buttons, completed count.
Hiring manager	`/jobs`, `/applications`, `/interviews` (their jobs only)	A to-do list: offers awaiting a decision, finished interviews, new applicants to screen; their pipeline; their jobs; coming up; newest applicants.
Admin	`GET /reports/progress?days=…`	Company totals, pipeline, progress by job (table), team workload (overdue scorecards flagged), results, scorecards, activity timeline; 7/30/90-day switch; “Email me this report”.

26.1 Accessible colours

The design used a palette validator that simulates colour blindness. It found that the green “Hired” and red “Rejected” colours look almost identical to people with red-green colour blindness. So outcomes always show an **icon and a label** (✓ Hired, ✗ Rejected), stage bars use one brand colour with numbers on each bar, and stage colours appear only as small dots beside text.

27. Frontend: styling, errors and security headers

27.1 Tailwind and design tokens

`app/globals.css` defines colours once as CSS variables (**design tokens**): `--primary: #3a36e0`, `--stage-hired`, `--status-warning`, … and `@theme inline` exposes them to Tailwind, so `bg-primary` or `text-status-good` work everywhere. Fonts are Bricolage Grotesque (headings) and Figtree (text), loaded with `next/font`.

27.2 Error and 404 pages

`app/not-found.tsx` is shown for unknown URLs; `app/error.tsx` (a client component with a `reset()` button) is shown if a page crashes, instead of a blank screen.

27.3 Security headers (next.config.ts)

Header	Protects against
Referrer-Policy: strict-origin-when-cross-origin	Invite links contain a secret token. This stops the full URL being sent to other sites (such as Google's script) as the "referrer".
X-Frame-Options: DENY	*Clickjacking*: another site showing HireDesk invisibly inside a frame to trick clicks.
X-Content-Type-Options: nosniff	Browsers guessing a file's type and running it as a script.
Permissions-Policy	Turns off camera, microphone, location and payment features the app never needs.

`poweredByHeader: false` also hides the "X-Powered-By: Next.js" header, giving attackers less information.

28. Deployment: how the app is hosted

Piece	Where	Details
Site	Vercel project `hiredesk-official`	Root directory `frontend`, framework Next.js, URL https://hiredesk-app.vercel.app. Env: `NEXT_PUBLIC_API_URL`, `NEXT_PUBLIC_GOOGLE_CLIENT_ID`.
API	Vercel project `hiredesk-api`	Root directory `backend`, framework FastAPI (found automatically at `app/main.py`), URL https://hiredesk-api.vercel.app. Env: all backend settings.
Database	Neon `hiredesk-db`	Added through the Vercel Marketplace and connected to `hiredesk-api`, which injects `DATABASE_URL` (pooled) and `DATABASE_URL_UNPOOLED` (direct).
Code	GitHub `Daveoski/hiredesk_official`	Both Vercel projects watch the `main` branch.

28.1 What each deployment file does

- `backend/vercel.json`: keeps tests and other development files out of the deployed function bundle.
- `backend/.python-version`: pins Python 3.12.
- `backend/requirements.txt`: runtime packages only; `requirements-dev.txt` adds pytest and httpx for tests.
- `frontend/.vercelignore`: keeps `node_modules`, build output and `.env` files out of CLI uploads.
- `frontend/bun.lock`: exact package versions; the npm lockfiles were removed so Vercel installs with Bun consistently.

28.2 Problems solved during deployment

Symptom	Real cause	Fix
Every build failed: “No Output Directory named public”.	Framework Preset was empty, so Vercel treated the Next.js app as a static site.	Set the preset to Next.js (through Vercel’s API).
`hiredesk-official.vercel.app` showed 404.	That short address belongs to someone else’s project; ours had a long auto-generated address.	Claimed `hiredesk-app.vercel.app` and `hiredesk-api.vercel.app`.
Visitors would have been asked to log in to Vercel.	Vercel Authentication protected every URL.	Production made public; preview deployments stay protected.
Database URL format.	Neon gives `postgres://…`; SQLAlchemy needs `postgresql+psycopg://…`.	A validator in `config.py` converts it.

Symptom	Real cause	Fix
Errors through the pooled connection.	Server-side prepared statements don't survive PgBouncer's transaction mode.	`prepare_threshold=None`.
Uploads over 4.5 MB.	Vercel's function body limit.	Direct signed uploads to Cloudinary.
Google "Access blocked: register the JavaScript origin".	The live address wasn't an authorized origin on the OAuth client.	Added in Google Cloud Console; verified by probing Google.
Invites never reached teammates.	Resend's test sender only delivers to the account owner.	Added Gmail SMTP sending + a guided setup script; invites now report `email_sent`.

29. CI/CD with GitHub Actions

CI (Continuous Integration) automatically checks every change. **CD (Continuous Delivery/Deployment)** automatically releases it. HireDesk uses GitHub Actions for CI and Vercel for CD: push to `main` → GitHub runs the checks and, at the same time, Vercel builds and deploys.

Workflow file	When it runs	What it does
backend.yml	Push or pull request touching `backend/`	Job **test**: starts a PostgreSQL 16 service, installs packages, runs all 73 tests, then `alembic check`. Job **migrate** (main only, after tests pass): `alembic upgrade head` against Neon using the secret `NEON_DATABASE_URL_UNPOOLED` (skips with a warning if the secret isn't set).
frontend.yml	Push or pull request touching `frontend/`	Installs with Bun (`--frozen-lockfile`), runs the TypeScript check and a production build.
progress-reports.yml	Every Monday 08:00 UTC, or by hand	Runs `python -m app.reports.send_progress_reports`. Needs the database secret plus Gmail (`SMTP_USERNAME`, `SMTP_PASSWORD`) or `RESEND_API_KEY`.

.github/workflows/backend.yml (shortened)

```
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:                      # a throwaway database just for the tests
        image: postgres:16
        env: { POSTGRES_DB: hiredesk_test, ... }
    steps:
      - uses: actions/checkout@v4                  # get the code
      - uses: actions/setup-python@v5              # install Python 3.12
      - run: pip install -r requirements-dev.txt
      - run: pytest -q
      - name: Models match migrations
        run: alembic check
  migrate:
    needs: test                                    # only after the tests pass
    if: github.ref == 'refs/heads/main'
    env: { DATABASE_URL: "${{ secrets.NEON_DATABASE_URL_UNPOOLED }}" }
    steps: [ ..., run: alembic upgrade head ]
```

Word	Meaning
Workflow	A YAML file describing automation.
Job / step	A workflow has jobs (run on fresh virtual machines); a job has steps (commands).
Runner	The machine GitHub provides (`ubuntu-latest`).
Service container	An extra program started for the job, here a PostgreSQL database.
Secret	An encrypted value stored in the repo settings, readable only by workflows (`secrets.NAME`).

Word	Meaning
`needs`	Run this job only after another succeeds.

> **Still to set up:** Add the GitHub secrets `NEON_DATABASE_URL_UNPOOLED` (and Gmail or Resend secrets for reports). Until then, database migrations must be run by hand, as was done for migration 0005.

30. External services setup

30.1 Google sign-in

1. A Google Cloud **OAuth client** (type “Web application”) gives the client ID `285740906915-…apps.googleusercontent.com`.
2. **Authorized JavaScript origins** must list every site that shows the button: `http://localhost:3000` and `https://hiredesk-app.vercel.app`.
3. If the consent screen is in *Testing*, only listed test users can sign in; *Publish* to allow everyone (HireDesk only asks for name and email).
4. The same client ID is in the frontend (`NEXT_PUBLIC_GOOGLE_CLIENT_ID`) and backend (`GOOGLE_CLIENT_ID`), and `GOOGLE_AUTH_ENABLED=true` on the backend.

30.2 Email

Either verify a domain in Resend, or use Gmail: turn on 2-Step Verification, create an **app password**, and set `SMTP_HOST=smtp.gmail.com`, `SMTP_PORT=587`, `SMTP_USERNAME`, `SMTP_PASSWORD`. The script `scripts/setup-google-signin-and-email.sh` walks through all of this, tests the password, sends a test invite, saves the settings to Vercel and redeploys.

30.3 Cloudinary

`CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY` and `CLOUDINARY_API_SECRET` on the backend. Files go in `hiredesk/cvs/` and `hiredesk/application-documents/` as *raw* resources.

31. What happened: the full timeline of the work

This is the order in which things were found, fixed and built. Use it to tell the story of the project.

31.1 First review and bug fixes

- **Backend crashed on start** in the project's virtual environment: `google-auth` and `alembic` were missing from `pyproject.toml`. Added with `uv add`; `requirements.txt` changed to `google-auth[requests]` because Google's verifier needs the `requests` package.
- **Development database was one migration behind** (0003) while the code read the new `password_login_enabled` column, so logins would have crashed. Ran `alembic upgrade head`.
- **Google sign-in errors vanished**: the API client redirected to /login on *any* 401, reloading the page. Now it only redirects when a session token was sent.
- **A test failed** because it checked the word "password" was absent from the response, and the new `password_login_enabled` field contains it. Tightened to `hashed_password`.

31.2 Google sign-in for admins and hiring managers

- Before: an invited hiring manager who used Google got a brand-new company and became its admin.
- After: existing users sign straight in; invited users join the inviting company with the invited role; a new company is only created from the Register page; the invite page's Google button checks the Google email matches the invite.

31.3 Password change, personal dashboards, admin emails and reports

- `PUT /auth/password` + Account page; confirmation email.
- Everyone lands on `/overview` with a role-specific dashboard.
- Admins emailed for every key event; `core/notifications.py` created.
- Progress report API, "Email me this report", and the weekly script.

31.4 Dashboard redesign

- Greeting banner, stat tiles, to-do lists, pipeline bars, agenda and timeline; entrance animations that respect reduced motion.
- Colour-blindness check on the stage colours (see 26.1).
- Visual check with real screenshots; found and fixed a mobile overflow (grid items needed `min-w-0`).

31.5 Deployment

- Backend made Vercel-ready (URL conversion, pooler setting, CORS regex, Python pin, function config); GitHub Actions workflow added.
- Direct-to-Cloudinary uploads for large files; weekly report workflow.

- Fixed the failing frontend project (framework preset), public access and short URL; created the API project, Neon database, environment variables and tables; connected frontend to backend; verified end to end.

31.6 Google origin and invitation emails

- Proved, by asking Google directly, that the live origin wasn't registered; it worked within 94 seconds of being added.
- Found that Resend's test sender can only reach its owner; added Gmail SMTP, an HTML invite email with a button, and `email_sent` reporting; wrote the guided setup script.

31.7 Full code review

Found	Fixed / added
Logout kept cached data (privacy).	Cache cleared when the account changes.
Scorecards accepted before the interview.	Server and UI wait until the interview starts; tests gained a `hold()` helper.
Dashboard scorecard links went to the wrong page.	They open the candidate page; Interviews page gets "Fill scorecard".
Past dates allowed in scheduling.	Date picker `min` + Zod rule.
Re-invite check could create duplicates; pending invites invisible; a 7-day lockout on re-inviting.	Correct check; pending list with Resend and Revoke; expired invitations reused.
Line breaks in names reached email subjects.	Whitespace normalised.
Duplicate unique rule on invitation tokens.	Migration 0005; `alembic check` in CI.
Sessions expired after 60 minutes and lost your place.	8 hours; return to the same page after login.
No 404/error pages, favicon or security headers; no frontend CI.	All added.

32. The hard problems and how they are solved

Hard problem	Solution	Where
Many companies and three roles share one database; data must never leak.	Every query starts from the user's *visible* set; out-of-scope = 404; role dependencies return 403.	auth/permissions.py, auth/dependencies.py
Two people schedule the same interviewer at the same time.	PostgreSQL exclusion constraint on interviewer + time range.	migration 0002, interviews/router.py
Two people move the same candidate at once.	`SELECT … FOR UPDATE` row lock + allowed-transition table.	candidates/service.py
Interview feedback copies whoever spoke first.	Scorecard privacy rule; immutable scorecards; no aggregate for interviewers.	scorecards/router.py
Passwords and invite links could be stolen from the database.	Argon2 password hashes; SHA-256 invite token hashes; one-time, expiring, rotating links.	core/security.py, users/router.py
The browser can't be trusted to say who it is with Google.	The server verifies Google's signed ID token, its audience and verified email.	auth/router.py
Files larger than the hosting limit.	Signed direct uploads, then server-side verification of the real size.	core/storage.py, lib/uploads.ts
Email outages shouldn't break hiring.	`send_email` returns a boolean, background tasks, and visible fallback links for invites.	core/email.py
Changes breaking production.	73 tests, `alembic check`, frontend build in CI before relying on a deploy.	.github/workflows/

33. Practice questions and model answers

Read the question, try to answer out loud, then compare.

Q. What is HireDesk in one sentence?

An applicant tracking system for small companies: admins build the team, hiring managers post jobs and decide, interviewers schedule and score, and candidates apply through a public link and get emails at every step.

Q. What is the tech stack?

Next.js/React/TypeScript with Tailwind, TanStack Query and Zustand on the frontend; Python FastAPI with SQLModel/SQLAlchemy, Pydantic and Alembic on the backend; PostgreSQL on Neon; Cloudinary for files; Resend or Gmail for email; Google Identity Services for sign-in; Vercel hosting; GitHub Actions for CI.

Q. Why separate frontend and backend?

The backend is the single source of truth for rules and permissions, which can't be trusted to the browser. Separating them lets each be deployed and scaled independently and keeps the API reusable.

Q. How does login work?

The user posts email and password to `/auth/login`. The server finds the user, checks the Argon2 hash, and returns a JWT signed with `JWT_SECRET` containing the user id and an expiry. The browser stores it and sends it as `Authorization: Bearer …`; `get_current_user` verifies it and loads the user on every request.

Q. What is a JWT and is it encrypted?

A JSON Web Token: header, payload, signature. It is *signed*, not encrypted: anyone can read the payload, but nobody can change it without the secret. That's why it only holds the user id and expiry, never secrets.

Q. Why hash passwords instead of encrypting them?

Encryption can be reversed with a key; hashing cannot. We never need the original password, only to check it, so a slow one-way hash (Argon2) is safest.

Q. What's the difference between authentication and authorization here?

Authentication is `get_current_user` (who are you, via the token). Authorization is `require_roles` (what your role can do) plus `permissions.py` (which rows you can see).

Q. Why return 404 instead of 403 for another company's candidate?

A 403 confirms the record exists. A 404 reveals nothing, which is safer in a multi-company system.

Q. How do you stop an interviewer being double-booked?

With a PostgreSQL exclusion constraint: no two *scheduled* rows may have the same interviewer and overlapping `tstzrange(starts_at, ends_at)`. Because the database enforces it, even simultaneous requests can't both succeed; we catch `ExclusionViolation` and return 409.

Q. Why not just check for overlaps in Python?

Race condition: two requests can both check, both see nothing, and both insert. Only the database can make the check and the insert one atomic operation.

Q. How do you keep the stage history consistent under concurrency?

`change_stage` reads the application with `SELECT … FOR UPDATE`, which locks the row until commit. A second request waits, then sees the new stage and is validated against it. A test fires parallel requests to prove it.

Q. What is a migration and why use Alembic?

A versioned script that changes the database structure. Alembic keeps the history so every environment (local, test, CI, Neon) reaches the same schema with `alembic upgrade head`, and `alembic check` catches models changed without a migration.

Q. What is the scorecard privacy rule and why?

An interviewer sees only their own scorecards until they've submitted all of theirs for that candidate; then they see others' submitted ones. It prevents anchoring bias so feedback stays independent. Interviewers never see the aggregate.

Q. How is the overall interviewer recommendation computed?

Any "not recommend" → not recommend; all "recommend" → recommend; any other mix → maybe. It's recalculated on every scorecard submission.

Q. How do invitations work and why store a hash?

The admin's invite creates a random token; only its SHA-256 hash is stored, and the link carries the token. Accepting hashes the token and looks for an open, unexpired invitation. Hashing means a database leak doesn't give working links. Resending rotates the token so old links die.

Q. How is Google sign-in verified?

The browser gets an ID token from Google and sends it to `/auth/google`. The server uses `google-auth` to verify Google's signature, the expiry and that the audience is our client ID, then requires `email_verified`. It never trusts an email from the browser.

Q. Why did Google say 'Access blocked' on the live site?

Google only allows its sign-in on registered JavaScript origins. The new Vercel URL wasn't listed on the OAuth client; adding it in Google Cloud Console fixed it.

Q. How do large CV uploads work on Vercel?

Vercel functions accept at most 4.5 MB per request. The backend signs a one-time Cloudinary upload for a random file id; the browser uploads directly; the application sends only the id; the backend checks the id format and asks Cloudinary for the real size, deleting oversized files.

Q. Why does `send_email` return a boolean instead of raising?

So an email outage never fails a business action like hiring. Most emails go in the background; invitations are sent immediately so the admin can be told and given the link to share.

Q. What is CORS and why does the API need it?

Browsers block a page from one origin calling an API on another unless the API allows that origin. The site and API are different origins on Vercel, so the API lists the site in `CORS_ORIGINS` and allows preview URLs by regex.

Q. What does TanStack Query give you over plain fetch?

Caching, loading and error states, automatic refetching after invalidation, and optimistic updates (the board moves a candidate instantly and rolls back if the server refuses).

Q. How does the frontend know who is logged in after a refresh?

Zustand's `persist` middleware saves the token and user in localStorage. The protected layout waits for it to rehydrate, then redirects to `/login?next=…` if there is no token.

Q. What privacy bug did you fix on logout?

The query cache survived logout, so another person signing in on the same browser could briefly see the previous user's data. Providers now clear the cache whenever the signed-in account changes.

Q. How are the tests organised?

pytest against a real PostgreSQL `_test` database migrated by Alembic; tables are emptied before each test. A `World` fixture builds a company with every role; `outbox` captures emails; `hold()` moves an interview into the past; `parallel()` tests concurrency. 73 tests in total.

Q. What does your CI/CD do on a push?

GitHub Actions runs the backend tests and `alembic check`, applies migrations to Neon on main (once the secret is set), and typechecks and builds the frontend. Vercel deploys both projects from `main` automatically.

Q. What is the difference between DATABASE_URL and DATABASE_URL_UNPOOLED?

The pooled URL goes through PgBouncer, good for many short serverless requests (with prepared statements disabled). The unpooled one is a direct connection, used for migrations.

Q. What would you improve next?

Rate-limit login, apply and upload; revoke existing sessions on password change; let admins remove team members; clean up uploads from abandoned applications; gate Vercel deploys on CI.

34. Known limitations and next steps

- **No rate limiting** on login, public apply and upload signing, so bots could try many times.
- **Tokens stay valid after a password change** until they expire (8 hours).
- **No way to remove or deactivate team members**, and no deletion/retention policy for candidate data and files.
- **Abandoned uploads** (signed but never used in an application) stay in Cloudinary.
- **Deploys don’t wait for CI**: Vercel deploys even if tests fail.
- **GitHub secrets not yet set**, so automatic migrations and Monday reports are inactive.
- **Email delivery to other people** needs the Gmail setup (or a verified Resend domain).

Appendix A. Every API endpoint

Method & path	Who	Purpose
GET /health	Anyone	API and database are up
POST /auth/register	Public	Create company + admin
POST /auth/login	Public	Email/password → token
POST /auth/google	Public	Google ID token → token (sign in, accept invite, or create company)
PUT /auth/password	Signed in	Change or set your password
POST /auth/accept-invite	Public (token)	Join a team with a password
GET /auth/me	Signed in	Who am I
POST /users	Admin	Invite a teammate
GET /users	Admin, HM	List team (optionally by role)
GET /users/invitations	Admin	Pending invitations
POST /users/invitations/{id}/resend	Admin	New link, 7 more days
DELETE /users/invitations/{id}	Admin	Revoke an invitation
GET /companies/me	Signed in	Your company
POST /jobs	HM	Create a job (draft)
GET /jobs	Admin, HM	List visible jobs
GET /jobs/{id}	Admin, HM	One job
PATCH /jobs/{id}	HM	Edit / open / close a job
GET /public/jobs/{id}	Public	Open job details
POST /public/jobs/{id}/uploads	Public	Sign a direct file upload
POST /public/jobs/{id}/applications	Public	Apply
GET /applications	Signed in (scoped)	List/filter/sort candidates
GET /applications/{id}	Signed in (scoped)	One candidate
PATCH /applications/{id}/stage	HM	Move forward one stage
PATCH /applications/{id}/assessment	HM	Match score and notes
POST /applications/{id}/decision	HM	Hire or reject
GET /applications/{id}/history	Admin, HM (scoped)	Stage history
GET /applications/{id}/scorecards	Signed in (scoped, privacy rule)	Scorecards (+ aggregate for managers)
POST /interviews	HM	Assign an interviewer

Method & path	Who	Purpose
GET /interviews	Signed in (scoped)	List interviews
GET /interviews/{id}	Signed in (scoped)	One interview
PATCH /interviews/{id}/schedule	Interviewer (own)	Set time and meeting details
POST /interviews/{id}/cancel	HM	Cancel; frees the slot
PUT /interviews/{id}/scorecard	Interviewer (own)	Submit the scorecard (after start)
GET /reports/progress	Admin	Company progress report
POST /reports/progress/email	Admin	Email the report to yourself

HM = hiring manager. “Scoped” means the results are limited by `permissions.py`.

Appendix B. HTTP status codes used

Code	Name	When HireDesk returns it
200	OK	Successful read or update.
201	Created	Registration, new job, new application, accepted invite, invitation created.
204	No Content	Invitation revoked.
400	Bad Request	Wrong current password, or new password same as old.
401	Unauthorized	Missing/invalid/expired token, wrong login, invalid Google token.
403	Forbidden	Your role can't do this; an invite accepted with a different Google email.
404	Not Found	Doesn't exist *or* isn't visible to you; job not open.
409	Conflict	Duplicate email/application, invalid stage move, interviewer busy, scorecard already submitted or too early.
410	Gone	Invitation expired, used or invalid.
413	Payload Too Large	File over the size limit.
422	Unprocessable Entity	Validation failed (bad email, short password, wrong file type, …).
502 / 503	Bad Gateway / Service Unavailable	File storage failed / is not configured; Google sign-in not configured.

Appendix C. Useful commands

Command cheat sheet

```
# Backend (from backend/)
python -m venv .venv  &&  pip install -r requirements-dev.txt     # set up
uvicorn app.main:app --reload                                      # run the API on :8000
alembic upgrade head                                               # apply migrations
alembic revision -m "describe change"                              # new migration file
alembic check                                                      # models vs migrations
pytest -q                                                          # run all tests
python -m app.reports.send_progress_reports --days 7               # send reports

# Frontend (from frontend/)
bun install            # install packages
bun run dev            # run the site on :3000
bun run typecheck      # TypeScript check
bun run build          # production build

# Deploy
git add . && git commit -m "message" && git push   # GitHub Actions + Vercel take over

# Setup Google sign-in + Gmail (Git Bash, from the repo root)
bash scripts/setup-google-signin-and-email.sh
```